

Large References, Fast Checks: Oracle-Guided Porting of a Stochastic Learning System

Anonymous authors

Paper under double-blind review

Abstract

Rewriting a program in another language is called porting. How do we check that the port preserves behaviour when the original gives different outputs for the same inputs, with no single prescribed correct output? We investigate a solution to this challenge while porting Metacat, written in Scheme, to Petacat, written in Python, using AI coding tools. Metacat makes analogies and learns from earlier attempts within an episode. It samples solutions from a relation between two countable sets: letter-string analogy problems and letter-string solutions. One problem can have several possible solutions, rather than the single output assigned by a deterministic function. Our testing process deliberately spends more up-front on collecting a reusable reference sample of solutions to analogy problems. Much smaller samples from the port then support fast, automated checks against that reference. Good-Turing analysis guides collection of the observed support set: the different solutions and other outcomes seen so far. The fast checks flag unexpected port solutions outside that set. A p50 head selects common reference outcomes accounting for at least half the sample; the checks also flag selected solutions missing from the port sample. These statistically motivated flags direct human attention to specific discrepancies instead of requiring repeated manual inspection of full solution distributions. They are reasons to investigate, not proof of defects. Our main result is practical: the process helped us locate and repair a port error and correct misleading test conditions. We began using the process before designing a formal study, so some historical build records are missing. Two later studies provide more precisely documented results, including failures of reference coverage that limit what the checks establish. Our second contribution is a novel testing process that combines established tools and applies them to Metacat’s episodic learning. We do not measure the speedup or claim complete behavioural equivalence or improved learning.

1 Introduction

If *abc* changes to *abd*, what should happen to *ijk*? Replacing the last letter with its successor gives *ijl*. Replacing it with *d* gives *ijd*. Both solutions follow an interpretation of the example; neither is the single prescribed solution. What does it mean for an analogy-making program with basic learning capabilities to produce a good solution?

Metacat explores that problem (Marshall, 1999; Marshall & Hofstadter, 1997). It extends Copycat (Mitchell, 1993; Hofstadter & Fluid Analogies Research Group, 1995), a computational model of analogy-making. For our testing problem, we can view it as a *fuzzy classifier* with an unknown set of solution classes and a simple learning loop.¹ It builds and assesses relationships between letters and groups, rather than choosing from a supplied list of solutions. Random choices influence what it explores, and memory of previous attempts can influence what it does next. Such a program is *stochastic*: another attempt at the same problem can give another result.

¹For a non-academic introduction, see Wikipedia, “Fuzzy classification”. This fuzzy-classifier description is a functional view of Metacat’s graded assessments, not a claim that solution frequencies are fuzzy-set membership values.

Our task was to rewrite Metacat from Scheme into Python, using AI coding tools, while preserving this behaviour. Such a rewrite is a *port*. We call the Scheme program the *reference program* and its Python port *Petacat*. A conventional test might require a particular output for each input. That expectation would miss the point of Metacat. Giving both programs the same random seed, the starting value for a pseudorandom number generator, does not solve the problem either. Different generators or a different order of random choices can lead the two programs along different paths.

We instead run the reference program extensively and retain the solutions it produces. That *reference sample* supports two checks after each change to the port: did an unexpected solution appear, and did a common reference solution fail to appear? Good–Turing guides collection of the observed support set, the different outcomes seen so far. The p50 rule selects common outcomes to look for. The resulting record is a testing *oracle*: a reference used to judge test results. It is not an authority on every possible solution. It can be incomplete, and building it can expose defects in the reference program itself.

The cost is deliberately asymmetric. We spend more once to build the reference sample so that later checks can use much smaller samples and report discrepancies automatically. A developer can investigate the flagged solutions instead of repeatedly inspecting full solution distributions. Fast iteration is the purpose of the process. We have not measured the speedup, and running fewer attempts does not imply the same reduction in elapsed time.

Memory makes the testing problem more interesting. A *run* is one attempt at a problem. An *episode* contains eight runs with memory retained between them and cleared before the next episode. We test experience-dependent behaviour by checking solutions selected by Metacat’s own quality and preference judgments. These *native* judgments let us test an aspect of conceptual coherence without prescribing one solution per run. We check the solutions that the program itself considers best, not every internal reasoning step. The comparison does not establish that memory improves learning.

The main result is practical: these checks helped improve the port. One discrepancy led us to repair how the port handled direction; another exposed unequal execution limits in the tests. We began using the process before designing a formal study, and historical records do not identify every tested build. Two later studies provide better-recorded evidence and expose further limits. Our second contribution is the testing process itself, including its application to episodic learning. The novelty lies in how we combine and apply established tools, not in inventing Good–Turing or p50. We explain the process first, then the repairs and the later studies.

2 Building the Oracle and Checking the Port

2.1 What We Record, and What We Check

We can state the distinction mathematically. Let $\mathcal{X}$ be the set of allowed letter-string analogy problems and $\mathcal{Y}$ the set of finite letter-string solutions. Both sets are countable: solutions are finite strings over a finite alphabet, and problems are specified by triples of such strings. For fixed program settings and initial memory, define a relation $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$: $(x, y) \in \mathcal{R}$ when the program can produce solution y for problem x with nonzero probability. A function from $\mathcal{X}$ to $\mathcal{Y}$ assigns exactly one solution to each problem. This relation need not: one problem can produce several solutions, and the same solution string can occur for several problems. The relation records what is possible; the sampling probabilities describe how often each result occurs. We leave the random seed out of the problem definition, rather than treating each seeded execution as a new problem.

An *outcome* is what we record from an attempt. Usually it is a solution string, such as `ijl`. The broader word matters because a run can also fail to produce a solution. Metacat works through small computational tasks called *codelets*. We limit their number per run and call that limit the *cap*. The historical records use `*NONE*` for stopping without a solution and `*CAP*` for reaching the cap. The later single-run study also uses `*ERROR*` for a failure inside the analogy program, or *engine*. These outcomes stay in the counts. Errors also raise a separate flag: the port is never required to reproduce a reference defect.

We count how often each outcome occurs, separately for each problem. For problem x , let $c_x(o)$ count outcome o among N_x reference runs. The observed support set and its estimated frequencies are

$$S_x = \{o : c_x(o) > 0\}, \quad \hat{p}_R(o | x) = c_x(o)/N_x.$$

Here S_x is the *observed support set for problem x* . It contains the outcomes we have actually seen. The *true support* contains every outcome with nonzero probability, including outcomes we have not yet seen. The reference probability $p_R(o | x)$ and port probability $p_P(o | x)$ depend on the program version, settings, memory, cap, and outcome definition. Recording letters also merges different internal structures with the same solution string. We therefore test the recorded solutions, not every conceptual route that could produce them.

How much reference collection is enough? Good–Turing (Good, 1953) estimates the chance that another run will produce an outcome type absent from the observed set. It uses *singletons*: outcome types seen exactly once. If five types have each appeared once in 100 observations, the singleton ratio is $5/100 = 0.05$. A solution ceases to be a singleton when it reappears; the solution is still in the observed set. Let $f_{1,x}$ count the singleton types. The unknown *missing mass*, meaning the total probability of outcomes outside our set, and its estimate are

$$M_{R,x} = \sum_{o \notin S_x} p_R(o | x), \quad \widehat{M}_{R,x} = f_{1,x}/N_x.$$

The estimate concerns the chance of a new type on another draw, not how many types remain undiscovered. Nor is it an upper confidence bound. A zero ratio says only that none of the observed types currently has count one. Stopping when the ratio falls below a threshold is a *heuristic*: a practical rule, not a guarantee of coverage. Guarantees for finite samples need additional uncertainty terms and assumptions (McAllester & Schapire, 2000). Our historical rule required a minimum sample; the later episodic rule allowed immediate stopping. Appendix A explains both rules and shows why a zero estimate cannot certify completeness.

Unexpected solutions are only half of the testing problem. We also want to notice solutions the port may have lost. For that purpose, the *p50 head* H_x selects common reference outcomes. Rank outcomes by decreasing count and take the shortest initial list accounting for at least half the observations. Break equal-count ties using fixed dictionary order. The dissertation’s alternative rules and analogy families (Marshall, 1999, Chapters 3 and 5) led us to expect many possible solutions; a fast check cannot reliably repeat every rare one. The p50 head focuses the missing-solution check on common outcomes.

For example, counts of 40 *ijl*, 25 *ijd*, 20 *ijk*, and 15 *ijm* give a p50 head of *ijl* and *ijd*: together they account for 65 of 100 observations. A port check producing *ijl*, *ijk*, and *ijn* misses head member *ijd* and produces unexpected solution *ijn*, which lies outside the observed reference support set. We shorten that description to an *outside solution*. If T_x is the observed port-check set, the reports are

$$\text{MISSING}_x = H_x \setminus T_x, \quad \text{NOVEL}_x = T_x \setminus S_x.$$

Set difference $\setminus$ means members of the first set absent from the second. **NOVEL** means outside the sample, not necessarily impossible for the reference. **MISSING** means absent from this check, not necessarily lost from the port. For example, a solution with port probability p is absent from n independent runs with probability $(1 - p)^n$. The p50 rule does not guarantee a minimum individual probability: with many equally likely solutions, even head members can be rare. A check with no flags means only that the selected common outcomes appeared and all sampled port outcomes were in the reference set. It does not certify the whole port. Appendix A.2 gives the sampling limits in detail.

2.2 Selecting Solutions From an Episode

What should we record when the program learns between attempts? The eight runs in an episode depend on earlier experience. Treating them as eight independent draws would ignore the memory we want to test.

Instead, we start each episode with cleared memory and treat the whole episode as one observation (Gerhold & Stoelinga, 2018). We then need a rule for selecting its recorded result; we call that rule a *projection*. The historical checks took the last solution found, a convenient endpoint rather than a justified optimum. The later study uses Metacat’s own assessments:

1. **best_a**, the *quality winner*, has the highest numerical solution-quality score.
2. **best_b**, the *preference winner*, is undefeated under Metacat’s pairwise conceptual-preference comparison. It need not have the highest quality.

Here *winner* means the solution selected by the stated rule. The program calls its numerical score *answer quality*; we use *solution quality* for the same score. These are the program’s judgments, not human correctness labels. Quality combines a rule assessment with temperature, an internal control quantity. The rule assessment considers consistency, generality, and economy of the rule; Appendix E gives the calculation. Preference supplies a different way to select among the episode’s interpretations.

Ties select the eligible solution reached first. Each complete episode that produces a solution contributes one string to each of two separate count tables. The *quality population* consists of the solutions that could win under the quality rule; the *preference population* is defined the same way for preference. We estimate discovery separately for each population. The two selected strings can coincide, and the populations are not assumed statistically independent. Distinct recorded types are distinct letter strings, even when different internal structures produce the same letters. Native judgments make our selection of episodic results internally justifiable. The resulting check addresses conceptual coherence through those selected solutions, not complete equivalence of internal reasoning.

Episodes with no solution, and incomplete episodes, remain in execution records but supply no winner. Coverage statements are therefore conditional on a complete episode producing a solution. Independently randomized episodes with the same initial memory, problem, and length can reasonably be modelled as draws from one population, provided no hidden state leaks between episodes. That assumption does not apply to inner runs pooled across changing memory. We do not compare complete solution sequences. The formal episode notation appears in Appendix A.3.

2.3 Freezing and Independent Validation

Once collection stops, *freeze* the observed support set and any p50 head: keep them unchanged during validation and port checks. Save the counts, seeds, settings, stopping reason, and outcome definition too. If we added every later discovery, we would erase the evidence that the original sample missed it. An outside or missing solution can reflect incomplete sampling, a reference defect, a port defect, or incompatible settings.

Can we trust the frozen set to cover most reference results? Run the reference again with new randomness and count observations outside the set. Each is a *validation miss*. A one-sided binomial upper confidence bound places a statistical upper limit on the chance of another miss, under the sampling assumptions. We call a set *qualified* if its bound meets the declared coverage target. Qualification concerns the reference set, not whether the port passes, and does not validate a p50 head. Failed and unperformed validations remain in the report.

The later episodic study targets an outside probability of at most 0.01: at least 99% of the reference’s selected solutions should fall inside the frozen set. That target is separate from the 10^{-4} construction heuristic. For simultaneous 95% confidence across 19 problems and two definitions, each bound receives error allowance $\alpha = 0.05/38$. In other words, we divide the total 5% error allowance among the 38 populations. This Bonferroni allocation does not require independence between definitions and remains fixed when some populations are not tested. With zero misses in m independent validation draws, the upper limit is $1 - \alpha^{1/m}$. At $m = 1000$ it is about 0.006611. The study validates only problems where both definitions freeze; either population can then qualify or fail separately.

The complete testing process is therefore: collect the reference sample, freeze its sets, assess coverage using independent validation, sample the port, and investigate the reported discrepancies. The history below did

not always include independent validation; the later studies show why that extra step matters. Reusing a reference makes checks economical, but not independent evidence. A developer can also adapt the port to the fixed sample, concealing problems that a fresh final audit would reveal.

3 How the Checks Improved the Port

We developed Petacat with AI coding tools based on language models, repeated comparison with the Scheme source, and consultation of the dissertation. The automated checks made that reading more focused. Instead of asking whether the whole port looked plausible, we could ask why a particular problem produced an unexpected solution.

The benchmark has 19 letter-string problems from demonstrations and simpler analogy cases (Appendix I). The historical reference was modified Metacat 1.2, running without its graphical interface, with a 100,000-codelet cap. Besides enabling this *headless* execution, modifications addressed unsupported object messages, whole-string rules, and dispatch; some can affect solutions. The reference is modified Metacat, not a claim of unchanged behaviour from the original release. Reconstruction patches are supplied without redistributing the original source.

Historical build provenance is incomplete: we did not retain enough records to identify every tested version and reproduce every intervention. The saved measurements can still be audited. The repair history remains the core engineering result, but the later studies cannot reconstruct missing historical evidence.

3.1 Following a Difference Into the Code

The saved single-run reference contains 374,500 runs. Across the benchmark, it records 366 different problem–outcome pairs and 27 p50 members. A pair identifies both the problem and its outcome: the same solution on two problems counts twice. Each fast check schedules 100 runs per problem, or 1,900 in total. Before repair, the check using the ordinary CPU (central processing unit) execution path flags five outside pairs: `eqe-baaab/abbbb`, `run6/cdddb`, `copy1/*NONE*`, `run1/*CAP*`, and `copy5/aac`. All p50 members appear, so the missing-solution check raises no flags.

Table 1: Observed flags, not estimated error rates. All cycles reuse the same seed block.

Archived cycle	Novel pairs	Missing head members
CPU before repairs	5	0
CPU after repairs	1	0
Archived MLX check	5	0

The first three flags led us to a direction-handling error, labelled RC-A in the development record. The program uses an *image*, an internal representation of letters or groups, when applying a rule. The port assembled the image of a leftward group in physical left-to-right order and labelled it rightward. The displayed letters could look unchanged because both the direction and the constituent order were reversed. A later operation that depended on direction exposed the mistake. Merely inspecting the displayed string would not necessarily reveal it.

The investigation combines a Python trace, Scheme-side probes, a repair, and focused regression tests. The development account reports that inserting the suspected defect into the reference reproduced `cdddb` and `*NONE*`. Its intervention table does not reproduce `abbbb`; evidence for that solution is the Python trace and disappearance after repair. We have not independently repeated those historical instrumented runs for this paper.

The `run1/*CAP*` flag, RC-B, had another explanation. The port initially stopped at 20,000 codelets, while the reference could use 100,000. The repaired driver reruns capped seeds with the larger budget. That flag disappeared when the caps were reconciled. The saved post-repair check includes 23 such reruns, or

1,923 attempts. Extending a seeded path by rerunning its seed at the larger cap is justified only if replay is deterministic and the larger cap does not affect earlier decisions. Matching numerical limits alone does not prove that a codelet means the same work in both programs.

After both repairs, the paired CPU replay has only `copy5/aac` outside. One proposed explanation, RC-C, was rejected; the solution remains unexplained. The separate check using the MLX numerical library still has five outside pairs, so the repairs are unvalidated on that execution path. All cycles reuse seeds 900,000–900,099 per input. Their before/after contrast is a paired replay, not independent confirmation, and a legitimate solution omitted from the reference could recur as a flag indefinitely.

What improved? The process located a semantic error, supported a repair, distinguished an unfair comparison from a program defect, and retained an unresolved discrepancy. The nominal reference-to-check ratio is about 197:1 in scheduled runs, excluding reruns and work per attempt. It describes the intended investment, not measured development savings.

3.2 What the Earlier Memory Checks Add

The earlier memory checks used the last solution found in each episode, which we call its *endpoint*. The historical reference is described as 500 eight-run episodes per problem; each port check uses 100 episodes. In the saved paired CPU checks, the number of outside problem–endpoint pairs falls from 17 before repair to 13 afterward. Their occurrences fall from 28 episodes to 14. Neither check misses a stored p50 member. The development investigation links the direction repair to four strings disappearing from the outside list, complementing the single-run evidence.

These records show how the checks were useful, but do not establish a controlled validation of learning behaviour. The historical episodic reference itself is absent from the supplement, so its size and outside labels cannot be independently verified from that attachment. The port cap was 20,000 against the reported reference cap of 100,000, and a capped run can affect every later run through memory. The episodic seeds also overlap the single-run seed block. Four episodes in each saved cycle produce no solution and are retained separately.

There is a concrete memory mechanism to test. Remembered solutions include their rules, so identical letters reached through different rules need not be duplicates. Both inspected solution-finding paths reject an already remembered structural solution and continue searching. Other memory and explanatory functions, their regression definitions, and the full historical tables are documented in Appendix B. Reading the code does not establish that both memory implementations behave identically when run. Those limitations motivated a later study with matching caps and solutions selected by native quality and preference.

4 A Recorded Study of Individual Runs

To provide more precisely documented evidence, we ran a later study, `support-v1a`. It records the program versions and every observation in order. Construction, validation, and port sampling use separate seed blocks. Every run starts with cleared memory and a direct 100,000-codelet cap. Each problem has 20,000 construction runs, 30,000 reference-validation runs, and 1,000 port runs: 969,000 observations across the benchmark. The full 20,000-run reference sample and its p50 head are frozen. Unlike the earlier adaptive collection, this study uses a fixed construction budget. We can inspect where Good–Turing would have stopped on the saved observations, but those retrospective calculations are not actual early stops.

The original collection stopped when the reference program failed reproducibly. We recorded a new protocol version that kept the same engines, budgets, seeds, and frozen sets, but counted recognized engine failures as `*ERROR*` before continuing to the next seed. Other failures still halt collection. Completed blocks from the original collection count once; replaying the interrupted block verified its earlier successful results. Exactly one observation is admitted per assigned seed. We did not repair the engine during collection. The continuation is a documented response to a failure, not a plan fixed in every detail before the study began.

4.1 What the Frozen References Found

Validation and port observations are divided into prespecified 100-run checks. Table 2 summarizes their results; Appendix C supplies protocols and per-problem tables.

Table 2: Versioned fresh-memory checks against 20,000 frozen construction runs per input. Counts pool 19 inputs for accounting, not to estimate a common outcome law. Errors are included in outside draws and also reported separately.

Measure	Reference validation	Port
Observations	570,000	19,000
100-run checks	5,700	190
Outside draws (including errors)	124	8
Checks with outside draws	119	6
Checks missing a p50 member	0	0
Engine-error observations	3	0

Fresh reference validation discovers outcomes outside the frozen sets on 16 problems. Its 124 outside observations include three reference errors on allowed inputs, with two exception signatures. Those failures remain in the denominators and receive separate hard-error flags. The reference campaign therefore stress-tests the original program as well as building an oracle. A large sample by another method might also have found these failures; they are not uniquely attributable to Good-Turing. Their seeds and diagnostic evidence are retained.

For `misc4`, `misc5`, and `copy6`, zero outside observations in 30,000 validation runs gives an individual one-sided 95% upper bound of about 9.9853×10^{-5} . None qualifies at 10^{-4} with simultaneous confidence across all 19 problems: even a zero-miss bound becomes about 1.9799×10^{-4} under that allocation.

The port produces eight outside observations representing five distinct problem–outcome pairs. Outcomes accounting for six of those eight observations also appear in fresh reference data. This overlap directly demonstrates that a frozen construction sample can omit legitimate reference outcomes. The remaining `misc1/*NONE*` and `copy5/acc` warrant investigation. No check misses a p50 member, and the port has no engine errors. Validation discoveries never enlarge the frozen oracle or erase its flags.

Construction and validation cost 950,000 reference executions. One fast check across the benchmark costs 1,900: the reference uses 500 times as many runs as one check. The ten recorded checks use 19,000 runs against one fixed port, not ten development revisions. Ordered-prefix analyses in Appendix C.3 examine hypothetical earlier stopping; they use different construction budgets and do not prove that one rule is cheaper for the same ability to detect discrepancies. Savings over the whole development process remain unmeasured.

5 A Recorded Study of Episodes With Memory

The episodic v3 study uses all 19 problems, eight runs per episode, a 100,000-codelet cap, and memory reset between episodes in both programs. It checks quality and preference winners against their frozen support sets. There is **no p50 head for these two definitions**; p50 was used in the single-run studies and historical endpoint checks.

We keep separate counts and calculate f_1/N for each winner population after every new episode. A population’s set freezes when its ratio first reaches 10^{-4} or less; the other population can keep accumulating solutions. Construction ends when both freeze or after 2,000 episodes. Four problems reuse complete 1,000-episode exploratory pilot samples. For those problems, the first freeze decision is at the end of the inherited sample, not at an earlier point chosen retrospectively. The other 15 permit immediate stopping. New first seeds are sampled independently with replacement, meaning a seed can be selected more than once; validation randomness is separate. The decision to reuse the pilot was made after exploration and is recorded as such.

Only problems where both sets freeze receive 1,000 reference-validation episodes. Every problem receives 100 port episodes. When reference coverage is unqualified, we still report what the port produced, without a coverage guarantee. The study retains 10,074 construction, 14,000 validation, and 1,900 port episodes: 25,974 episodes and 207,791 actual inner runs. A pilot error prevents one scheduled inner run. No construction was extended, validation enlarged, or frozen set updated after the results.

5.1 What Qualified, and What Did Not

Only `copy1`, `copy2`, and `copy3` meet the limit of 1% outside solutions for both definitions. Eight of the 28 tested populations qualify: those six, `misc3 best_a`, and `copy4 best_b`. Twenty fail. Ten populations belonging to five construction-limited problems receive no validation. Each qualified population has zero misses in 1,000 validation episodes with solutions, giving the simultaneous-confidence upper bound 0.006611. Appendix D reports every problem, including failed and skipped validation and port results against unqualified sets.

Why did so many sets fail? The `misc5` problem shows one danger clearly. Both sets freeze after only two episodes: neither set contains a singleton, so both estimates are zero. Yet fresh validation produces 480 quality winners and 591 preference winners outside those sets in 1,000 episodes. The early estimate was zero, but the reference coverage was poor. Independent validation exposed the failure. This early stopping is not evidence that learning made collection more efficient.

Could a different population work better? Possibly. Quality and preference make the selected episode result internally justifiable by Metacat, unlike simply taking the last solution. We did not know that these populations would suit Good-Turing-guided collection. Another conceptually meaningful projection, or rule for selecting episodic results, might let Good-Turing guide collection more effectively and produce a better oracle. It needs a new experiment and independent validation. These data cannot separate population choice from immediate stopping, and changing the projection cannot make a zero singleton ratio a confidence certificate.

Among qualified populations, only `misc3 best_a` has outside port winners: 24 of 100. The other seven have none. Large counts against failed references, such as 47 quality and 62 preference winners for `misc5`, remain descriptive. Five `copy5` validation episodes and four port episodes produce no solution, leaving winner denominators of 995 and 96; the other validation and port denominators are 1,000 and 100. The incomplete construction episode supplies no winner. All remain in execution accounting (Appendix F). Comparing selected winners does not test whether the programs produce a solution or terminate equally often.

5.2 Looking Closely at `misc3`

The problem is `abc -> aabbcc`; `kkjjii -> ?`: the example doubles letters, while the target already contains pairs in descending alphabetical order. Both frozen sets contain `kji`, `kkjjii`, and `kkkjjjiii`, representing one, two, and three copies of each target letter. Quality freezes after 11 episodes with counts 3, 6, and 2; preference after seven with counts 2, 3, and 2. Identical observed sets do not mean identical winner populations.

Table 3: Frozen-set results for `misc3`. Denominators count complete episodes with solutions. Bounds use the fixed 38-population confidence allocation. Only the quality reference qualifies at the 0.01 target; neither row is a verdict on the port.

Winner	Freeze episodes	Ref. misses / episodes	Upper bound	Port outside / episodes
Quality (<code>best_a</code>)	11	0 / 1,000	0.006611	24 / 100
Preference (<code>best_b</code>)	7	99 / 1,000	0.130501	26 / 100

The 24 outside port quality winners comprise ten strings; the 26 preference winners comprise 16. Every outside port string also occurs among the reference validation’s selected preference winners. Metacat can produce them. The quality discrepancy concerns their selection as winners, not whether those letter strings are possible at all. Failed preference coverage does not invalidate the separately qualified quality reference.

Could tied scores explain the discrepancy? Of the 24 quality events, 21 have one maximum-quality string, one ties among outside strings, and two tie between outside and inside strings. Only the last two could change membership under another tie rule. We retain the specified earliest-occurrence rule. Nine outside episodes produce a solution on all eight runs, so execution caps cannot explain the whole finding either.

Reading the code reveals two differences in *rule generalization*, which turns specific changes into a more general rule. First, the port excludes some shared changes that the reference retains unless the change has an explicitly named relation. Second, the port omits some reference checks on when changes to components can be generalized. These are two candidate explanations, not two established root causes. The saved winner descriptions lack the rule clauses and internal workspace details needed to assign particular episodes to either explanation. We have not repaired the engine or removed or reclassified observations in response to these findings. Appendix E gives the complete counts, diagnostic groups, and limits of the source and raw-record audits.

For the `misc3` quality set, 11 construction plus 1,000 validation episodes cost 8,088 reference runs; the port check costs 800. The check has given us a specific discrepancy to investigate. We do not yet know whether memory caused it: that investigation would need a matched experiment with memory disabled as well as enabled.

6 Relation to Earlier Work

The difficulty of deciding what a test result should be is the established *testing oracle problem* (Weyuker, 1982; Barr et al., 2015; Patel & Hierons, 2018). Using another implementation as a fallible reference also has precedents in *pseudo-oracles* (Davis & Weyuker, 1981) and *differential testing* (McKeeman, 1998). *Metamorphic testing* checks relationships between executions instead of prescribing each output (Chen et al., 1998; 2018; Segura et al., 2016). It could complement our checks when such relationships are known.

The closest precedent for using discovery statistics is STADS (Böhme, 2018), which treats program behaviours as species discovered through sampling. Related work estimates remaining risk in *fuzzing*, testing with generated inputs (Böhme et al., 2021). We do not claim to have invented discovery statistics in software testing. The p50 head is an empirical smallest covering region (O’Neill, 2022): a minimal set containing a specified share of observed results. Neither its selection algorithm nor the 50% choice is claimed to be new or optimal. Resetting probabilistic systems between tests (Gerhold & Stoelinga, 2018) also precedes our episode sampling. Results for stationary Markov sequences (Pananjady et al., 2024) rely on explicit assumptions; they do not automatically apply to runs whose memory changes during an episode.

Our process contribution combines these ideas for a particular development task: construct and freeze a reference, use fast checks to name unexpected and missing solutions, and investigate those solutions to improve a stochastic port. We also apply the construction and support checks to solutions selected by native episodic judgments. The contribution includes what this testing process accomplished and where it failed. Appendix J discusses further statistical and testing precedents; their guarantees do not automatically transfer to this sampling scheme.

7 Limits and Interpretation

This investigation is an engineering case study. It does not establish, with a calibrated error rate, that the two programs can produce exactly the same outcomes. The historical repairs are central, but their incomplete build records remain a limitation. The later studies evaluate recorded, fixed versions; they do not follow a new sequence of repairs. We have not measured detection across a controlled collection of defects, compared alternative methods with equal opportunities to reuse a reference, or timed the full development process.

The reference cost can be spread over many checks, but the whole cost also includes validation, sampling the port, investigating flags, and repairs. A flag-free check still leaves rare defects and unobserved conceptual differences possible. Changing caps, episode length, initial memory, or outcome definitions changes what is being tested and requires a new reference or a justified transformation. Confirmed reference defects invalidate the affected reference behaviour.

All 19 problems come from one cognitive architecture. Study choices were made during development rather than fully specified before exploration. Tests with memory retained and cleared, and tests with known memory defects, would help establish which learning-related changes the process detects. Matching solution strings does not establish identical reasoning. We claim neither better learning nor demonstrated transfer to other problem domains, continuous outputs, or systems whose generation process keeps changing. The evidence supports a useful testing process and concrete investigations, within those limits.

8 Potential Application of the Process

Must this testing process stop at porting? No. The investment resembles *test-driven development* (TDD): spend time up-front writing an automated test, then use its reports while developing the implementation instead of inspecting every passing result.² For a stochastic system, first decide what counts as a distinct outcome, what one observation represents, and how much sampling to do. Build the reference sample, freeze it, validate its coverage, and let automated flags direct further work. Passing results need not each demand human inspection. A sampled reference guards observed behaviour, not new capabilities; those still need requirements or other tests.

The reference could even be human. Consider speeches from one consenting speaker. Define distinguishable phrasing patterns before testing, so a new sentence is not automatically a new pattern. A p50 head selects patterns accounting for at least half their recorded occurrences; a Good–Turing-like discovery estimate, adapted to dependencies within and between speeches, guides collection. Held-out speeches assess coverage. A fine-tuned language model could then be checked for missing characteristic expressions and patterns outside the observed support, without prescribing their repetition rates. Outside patterns prompt inspection, not proof the speaker could never use them. This speaker-style test is a proposed experiment, not a result of this study. Appendix J.1 gives additional software-development examples.

9 Conclusion

The practical result is a better-directed development process: collect extensively, check quickly, and investigate the named discrepancies. Good–Turing guides the reference used to flag unexpected solutions; p50 selects common solutions whose absence deserves attention. The checks helped us repair the port and correct misleading test conditions.

Applying the process to episodic solutions brings the system’s simple learning loop into the test. We check solutions selected by native quality and preference, without requiring one fixed solution per run. The later studies document both useful findings and important failures of reference coverage. Those failures are part of the result, not evidence to discard. The statistical ingredients are established; combining them into a reusable testing process, and applying that process to episodic learning, is our second contribution. The process helps us find concrete problems to investigate. It does not certify everything the port can do.

²See Martin Fowler’s practitioner account, “Test Driven Development”.

References

- Andrea Arcuri and Lionel Briand. A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability*, 24(3):219–250, 2014. doi: 10.1002/stvr.1486. URL <https://doi.org/10.1002/stvr.1486>.
- Robert Axtell, Robert Axelrod, Joshua M. Epstein, and Michael D. Cohen. Aligning simulation models: A case study and results. *Computational and Mathematical Organization Theory*, 1(2):123–141, 1996. doi: 10.1007/BF01299065. URL <https://doi.org/10.1007/BF01299065>.
- Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785. URL <https://doi.org/10.1109/TSE.2014.2372785>.
- Marcel Böhme. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology*, 27(2):7:1–7:52, 2018. doi: 10.1145/3210309. URL <https://doi.org/10.1145/3210309>.
- Marcel Böhme, Danushka Liyanage, and Valentin Wüstholtz. Estimating residual risk in greybox fuzzing. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 230–241, 2021. doi: 10.1145/3468264.3468570. URL <https://doi.org/10.1145/3468264.3468570>.
- Anne Chao. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, 11(4):265–270, 1984.
- Anne Chao and Shen-Ming Lee. Estimating the number of classes via sample coverage. *Journal of the American Statistical Association*, 87(417):210–217, 1992. doi: 10.1080/01621459.1992.10475194. URL <https://doi.org/10.1080/01621459.1992.10475194>.
- Tsong Yueh Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, 1998. URL <https://arxiv.org/abs/2002.12543>. Reissued on arXiv in 2020.
- Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018. doi: 10.1145/3143561. URL <https://doi.org/10.1145/3143561>.
- Martin D. Davis and Elaine J. Weyuker. Pseudo-oracles for non-testable programs. In *Proceedings of the ACM ’81 Conference*, pp. 254–257, 1981. doi: 10.1145/800175.809889. URL <https://doi.org/10.1145/800175.809889>.
- Saikat Dutta, Owolabi Legunsen, Zixin Huang, and Sasa Misailovic. Testing probabilistic programming systems. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 574–586, 2018. doi: 10.1145/3236024.3236057. URL <https://doi.org/10.1145/3236024.3236057>.
- Bradley Efron and Ronald Thisted. Estimating the number of unseen species: How many words did Shakespeare know? *Biometrika*, 63(3):435–447, 1976. doi: 10.1093/biomet/63.3.435. URL <https://doi.org/10.1093/biomet/63.3.435>.
- Stephen G. Eick, Catherine R. Loader, M. David Long, Lawrence G. Votta, and Scott Vander Wiel. Estimating software fault content before coding. In *Proceedings of the 14th International Conference on Software Engineering (ICSE)*, pp. 59–65, 1992. doi: 10.1145/143062.143090. URL <https://doi.org/10.1145/143062.143090>.
- William A. Gale and Geoffrey Sampson. Good–Turing frequency estimation without tears. *Journal of Quantitative Linguistics*, 2(3):217–237, 1995. doi: 10.1080/09296179508590051. URL <https://doi.org/10.1080/09296179508590051>.

- Marcus Gerhold and Mariëlle Stoelinga. Model-based testing of probabilistic systems. *Formal Aspects of Computing*, 30(1):77–106, 2018. doi: 10.1007/s00165-017-0440-4. URL <https://doi.org/10.1007/s00165-017-0440-4>.
- I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953. doi: 10.1093/biomet/40.3-4.237. URL <https://doi.org/10.1093/biomet/40.3-4.237>.
- Ralph Guderlei and Johannes Mayer. Statistical metamorphic testing: Testing programs with random output by means of statistical hypothesis tests and metamorphic testing. In *Seventh International Conference on Quality Software (QSIC)*, pp. 404–409, 2007. doi: 10.1109/QSIC.2007.4385527. URL <https://doi.org/10.1109/QSIC.2007.4385527>.
- Douglas R. Hofstadter and Fluid Analogies Research Group. *Fluid Concepts and Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. Basic Books, 1995. ISBN 9780465051540.
- Axel Legay, Benoît Delahaye, and Saddek Bensalem. Statistical model checking: An overview. In *Runtime Verification (RV)*, volume 6418 of *Lecture Notes in Computer Science*, pp. 122–135. Springer, 2010. doi: 10.1007/978-3-642-16612-9_11. URL https://doi.org/10.1007/978-3-642-16612-9_11.
- Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 643–653, 2014. doi: 10.1145/2635868.2635920. URL <https://doi.org/10.1145/2635868.2635920>.
- James B. Marshall. *Metacat: A Self-Watching Cognitive Architecture for Analogy-Making and High-Level Perception*. PhD thesis, Indiana University, Bloomington, 1999. URL <https://science.slc.edu/jmarshall/metacat/dissertation.pdf>.
- James B. Marshall and Douglas R. Hofstadter. The Metacat project: A self-watching model of analogy-making. *Cognitive Studies: Bulletin of the Japanese Cognitive Science Society*, 4(4):57–71, 1997. doi: 10.11225/jcss.4.4_57. URL https://doi.org/10.11225/jcss.4.4_57.
- David A. McAllester and Robert E. Schapire. On the convergence rate of Good–Turing estimators. In *Proceedings of the 13th Annual Conference on Computational Learning Theory (COLT)*, pp. 1–6, 2000. URL <https://www.learningtheory.org/colt2000/papers/McAllesterSchapire.pdf>.
- William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998. URL https://www.bitsavers.org/pdf/dec/dtj/dtj_v10-01_1998.pdf.
- Melanie Mitchell. *Analogy-Making as Perception: A Computer Model*. MIT Press, 1993. URL <https://mitpress.mit.edu/9780262132893/analogy-making-as-perception/>.
- Ben O’Neill. Smallest covering regions and highest density regions for discrete distributions. *Computational Statistics*, 37(3):1229–1254, 2022. doi: 10.1007/s00180-021-01172-6. URL <https://doi.org/10.1007/s00180-021-01172-6>.
- Alon Orlitsky, Ananda Theertha Suresh, and Yihong Wu. Optimal prediction of the number of unseen species. *Proceedings of the National Academy of Sciences*, 113(47):13283–13288, 2016. doi: 10.1073/pnas.1607774113. URL <https://doi.org/10.1073/pnas.1607774113>.
- Ashwin Pananjady, Vidya Muthukumar, and Andrew Thangaraj. Just wing it: Near-optimal estimation of missing mass in a Markovian sequence. *Journal of Machine Learning Research*, 25(312):1–43, 2024. URL <https://jmlr.org/papers/v25/24-0511.html>.
- Krishna Patel and Robert M. Hierons. A mapping study on testing non-testable systems. *Software Quality Journal*, 26(4):1373–1413, 2018. doi: 10.1007/s11219-017-9392-4. URL <https://doi.org/10.1007/s11219-017-9392-4>.

- Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. doi: 10.1109/TSE.2016.2532875. URL <https://eprints.whiterose.ac.uk/id/eprint/110335/>.
- Elaine J. Weyuker. On testing non-testable programs. *The Computer Journal*, 25(4):465–470, 1982. doi: 10.1093/comjnl/25.4.465. URL <https://doi.org/10.1093/comjnl/25.4.465>.
- Uri Wilensky and William Rand. Making models match: Replicating an agent-based model. *Journal of Artificial Societies and Social Simulation*, 10(4), 2007. URL <https://www.jasss.org/10/4/2.html>. Article 2.
- Shin Yoo. Metamorphic testing of stochastic optimisation. In *Third International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pp. 192–201, 2010. doi: 10.1109/ICSTW.2010.26. URL <https://doi.org/10.1109/ICSTW.2010.26>.
- Håkan L. S. Younes and Reid G. Simmons. Probabilistic verification of discrete event systems using acceptance sampling. In *Computer Aided Verification (CAV)*, volume 2404 of *Lecture Notes in Computer Science*, pp. 223–235. Springer, 2002. doi: 10.1007/3-540-45657-0_17. URL https://doi.org/10.1007/3-540-45657-0_17.

A Sampling Rules and Statistical Details

A.1 Historical Good–Turing Collection

The historical single-run collector used these rules:

1. After at least 3,000 runs, stop if there is a singleton and $f_{1,x}/N_x \leq 10^{-4}$, or 0.0001.
2. If there are no singletons, require at least 10,000 runs before stopping.
3. Record an overshoot below 6×10^{-5} , but do not treat it as a second acceptance condition. Use a 50,000-run checkpoint and a 200,000-run hard maximum to limit spending.

Eight worker processes sampled separate, interleaved seed indices. A coordinator merged their counts and calculated one overall singleton ratio; workers did not make independent stopping decisions. Work already underway could finish after a stopping condition was reached.

The 10,000-run minimum sounds substantial, but it does not certify the target. Suppose there are just two outcomes, with probabilities 0.9998 and 0.0002. The chance of seeing only the first in 10,000 runs is

$$(0.9998)^{10000} \simeq 0.1353.$$

In that case, the estimate is zero, although the actual missing mass is 2×10^{-4} , twice the intended target. Also, one singleton at 10,000 runs satisfies the positive-singleton rule; the minimum is not justified by rejecting that case.

The historical files contain counts and codelet-use summaries but no discovery order, so they cannot support a retrospective comparison of stopping rules. The later single-run study preserves order and permits the comparisons in Section C.3.

The later episodic study deliberately allowed an earlier stop: check after every episode, with no minimum, and *freeze* the observed set as soon as $f_1/N \leq 10^{-4}$. If the first two episodes select the same solution, collection can stop with a one-solution set. We call the frozen set a *candidate oracle*. Fresh reference observations then test whether that set covers enough reference behaviour to meet the declared target.

A.2 What the Flags Can and Cannot Tell Us

A flag naming a solution gives us a concrete case to investigate. Interpreting it still requires care about sampling, reference coverage, and the chance of observing the solution within the check budget. This section states those limits.

A.2.1 When a Common Solution Is Missing

Suppose a frozen head contains a solution with port probability $p_P(o \mid x)$. If the check observations are independent and that probability stays fixed, the chance of missing it in all n_x observations is

$$\Pr(o \notin T_x \mid S_x, H_x) = (1 - p_P(o \mid x))^{n_x}.$$

This expression multiplies the probability of not seeing the solution on one draw across all draws, using the *port's* probability. Substituting the observed reference share $\hat{p}_R(o \mid x)$ gives a comparison number we call a *plug-in baseline*: an estimate plugged into a formula as if it were the true probability. The substitution ignores uncertainty in the reference sample, head selection, and stopping rule, and a reference share is not an established lower bound on the port's chance of producing that solution. The baseline is therefore not a calibrated error probability.

If we could justify a lower bound $p_P(o \mid x) \geq \ell_{x,o} > 0$, then $(1 - \ell_{x,o})^{n_x}$ would be an upper bound on absence. Adding these bounds over all selected solutions gives a union bound for a check cycle; the absence events

themselves need not be independent. Establishing such lower bounds, with their uncertainty, is separate work; support equality alone supplies none.

There is also a difference between a false alarm and a missed defect. Failing to see a still-possible solution can produce a misleading discrepancy flag. If the port has completely lost a solution, so that $p_P(o \mid x) = 0$, then the solution is absent with certainty. Its head-membership test cannot miss that deletion merely because the check sample is small.

A.2.2 When a Solution Falls Outside the Reference

Outside solutions have three useful counts. How many occurrences were there? How many different solution types appeared? Did the problem receive any outside-solution flag at all? Ten occurrences of one solution are ten draws, one distinct type, and one flagged problem.

For frozen S_x , let $q_{P,x} = \sum_{o \notin S_x} p_P(o \mid x)$ be the port's probability of producing anything outside the reference. With independent check draws, the expected number D_x of outside draws and the probability of at least one are

$$\mathbb{E}[D_x \mid S_x] = n_x q_{P,x}, \quad \Pr(D_x > 0 \mid S_x) = 1 - (1 - q_{P,x})^{n_x}.$$

The test software also lists distinct problem–outcome pairs. If U_x counts the distinct outside types for problem x , then

$$\mathbb{E}[U_x \mid S_x] = \sum_{o \notin S_x} [1 - (1 - p_P(o \mid x))^{n_x}] \leq n_x q_{P,x}.$$

Each term is the chance of seeing that outside type at least once. Counting distinct types cannot exceed counting every occurrence.

Reference validation concerns $M_{R,x}$, the chance of an unseen solution from the reference. It does not bound $q_{P,x}$, the port's chance of an outside solution. The port check therefore records outside solutions for investigation without treating the reference's coverage bound as a guarantee about the port.

A budgeted check can also miss rare defects. Suppose the solutions caused by a defect have total port probability ϵ and all lie outside S_x . The chance of missing all of them in n_x independent runs is $(1 - \epsilon)^{n_x}$; with 100 runs, detection probabilities for $\epsilon = 10^{-4}, 10^{-3}, 10^{-2}$ are about 0.0100, 0.0952, and 0.6340. Seeing all selected common solutions does not make those rare defects easy to detect.

A.3 Episode Notation

Let $\mathcal{E}_{x,e}$ denote episode e , including its solutions and memory changes, and let $Z_{x,e} = \phi(\mathcal{E}_{x,e})$ be its selected result under the chosen projection ϕ . Episode counts and observed sets follow the same pattern as run counts:

$$c_x^\phi(z) = \sum_{e=1}^{N_x} \mathbf{1}\{Z_{x,e} = z\}, \quad S_x^\phi = \{z : c_x^\phi(z) > 0\}.$$

The indicator $\mathbf{1}$ adds one when the selected result equals z and zero otherwise. Here N_x counts episodes rather than the LN_x runs inside them; for quality and preference winners, it counts complete episodes with a solution. Episodes with no solution and program errors stay in separate records, so every coverage statement is conditional on completing an episode with a solution.

A.4 Counting the Whole Cost

Let C_R be the one-time reference cost. For check cycle k , let $C_{P,k}$ be the sampling-and-comparison cost and $C_{F,k}$ the cost of investigating its findings. Across K cycles that can legitimately reuse the reference, the total and average costs are

$$C_{\text{total}}(K) = C_R + \sum_{k=1}^K (C_{P,k} + C_{F,k}), \quad \overline{C}(K) = \frac{C_R}{K} + \frac{1}{K} \sum_{k=1}^K (C_{P,k} + C_{F,k}).$$

The first term in the average, C_R/K , spreads the reference cost across checks; that is what *amortized cost* means here. It is an accounting identity. Any fair comparison with another method must let that method reuse its reference too.

In a single-run check the imbalance is N_x reference runs against n_x port runs, usually with $N_x \gg n_x$; for learning-mode tests the units are episodes. Counts are not elapsed time. One episode may cost far more than another, especially when memory changes how long finding a solution takes, and Scheme versus Python, processor, numerical software, and concurrency all move the cost. We therefore report count ratios only as count ratios.

What can stay fixed while the port changes? We retain reference counts, the observed set, any selected head, program and dependency versions, seeds, problem schedules, initial memory, stopping conditions, and outcome definitions. Together, these specify what behaviour the reference describes. Changing an episode from eight runs to sixteen changes that specification, as does changing the initial memory, the cap, or the meaning of an outcome; each needs a new reference or a justified transformation of the old one. A confirmed reference defect invalidates the affected part of the reference.

There is another limit to reuse. Developers can gradually adapt a port to the particular reference they keep checking. Repeated checks are then not independent new evidence, and their individual statistical claims do not add up to a guarantee over the whole development history. Fresh final audit samples and tests with known defects remain important.

B Historical Evidence and Memory Behaviour

B.1 What the Historical Records Establish

The historical reference was a modified Metacat 1.2 with a 100,000-codelet cap, run *headless*, without the graphical interface. The modifications also addressed unsupported messages sent to program objects, rules acting on whole strings, and message dispatch, and some of them can affect solutions. This reference is therefore modified Metacat, and we do not claim the modifications preserve every aspect of its behaviour.

A reconstruction package supplies patches for the linked original release without redistributing its source. Reconstructing the current modified source still does not identify the exact build behind every historical sample; that evidence is missing. The old counts can be audited. The complete historical experiment cannot be reproduced end to end.

The historical single-run checks use seeds 900,000–900,099 for each input. Because the seeds are reused before and after repairs, the two checks are a paired replay.

B.2 The Large Historical Reference

The saved reference contains 374,500 runs and 366 distinct problem–outcome pairs. The same solution to two different problems counts as two pairs. There are 27 selected p50 head members across the 19 problems. Reference sizes range from 10,250 to 51,000 runs, averaging 19,710.5 per problem. Six problems have no singletons; three have singleton ratios above 10^{-4} .

Table 4 gives the full counts. In its headings, $|S_x|$ is the number of different observed outcomes and $|H_x|$ is the number selected for the p50 head. A stored stopping label records what the collector reported.

Table 4: Archived reference counts. Stopping labels describe the heuristic, not certified completeness.

Input	N_x	$ S_x $	$f_{1,x}$	$f_{1,x}/N_x$	Stored reason	$ H_x $
misc4	11,000	4	0	0.00000000	no_singletons	2
fig5.7	18,125	18	1	0.00005517	saturated	2
misc3	19,000	54	4	0.00021053	shards_exited	2
misc5	11,125	8	1	0.00008989	saturated	3
misc2	11,000	6	1	0.00009091	saturated	1
run1	11,000	11	0	0.00000000	no_singletons	1
run4	11,000	10	1	0.00009091	saturated	2
misc1	30,750	18	3	0.00009756	saturated	1
fig5.4-top	51,000	61	19	0.00037255	checkpoint	2
ege-baaab	51,000	51	13	0.00025490	checkpoint	1
run6	47,500	80	4	0.00008421	saturated	1
run3	21,000	12	2	0.00009524	saturated	2
run2	11,000	10	1	0.00009091	saturated	1
copy1	10,875	2	0	0.00000000	no_singletons	1
copy2	11,250	1	0	0.00000000	no_singletons	1
copy3	11,000	1	0	0.00000000	no_singletons	1
copy4	11,125	2	0	0.00000000	no_singletons	1
copy5	15,500	13	1	0.00006452	saturated	1
copy6	10,250	4	1	0.00009756	saturated	1

saturated means only that the heuristic stopped collection. Two checkpoint cases finished at 51,000 runs because work was already underway. The other above-target case, **misc3**, records **shards_exited**; development notes describe a manual stop, but the aggregate file does not say why the worker processes exited. The spread of collection sizes makes adaptive spending worth considering, though not yet proven better than a fixed budget.

The historical probability calculations are worth keeping, with their assumptions stated. With 100 runs per check, the smallest head-member reference share is $4089/21000 = 0.194714\dots$ for solution `wyz` on problem `run3`. The absence formula gives 3.9355×10^{-10} for it, and the 27 member-specific absence terms add to 6.1340×10^{-10} . Both are plug-in baselines in the sense of Section A.2.1.

The saved singleton ratios give 0.169485 expected outside draws per benchmark-wide cycle and 0.167898 expected problems with at least one outside draw. If checks are independent across problems too, the plug-in probability of any outside draw is $1 - \prod_x (1 - \widehat{M}_{R,x})^{100} = 0.155914$. The expected number of distinct outside pairs is bounded above by the expected draw count. A zero singleton ratio contributes zero to these calculations without establishing zero risk.

The rough design figure $19 \times 100 \times 10^{-4} = 0.19$ is therefore no proved upper bound: three recorded estimates exceed the target, and none is an upper confidence limit. The plug-in total happens to fall below 0.19.

B.3 Making the Execution Limits Comparable

One historical check schedules 100 initial runs per problem, or 1,900 across the benchmark, so the reference-to-check ratio is $374500/1900 = 197.105\dots$ in run counts.

The port initially used a working cap of 20,000 codelets against 100,000 in the reference, so a port run could stop before the reference’s budget was exhausted. The repaired test driver reruns capped seeds at 100,000 codelets; the saved post-repair cycle has 23 such reruns, for 1,923 execution attempts. The 197-fold ratio excludes those attempts and any difference in work per attempt.

Rerunning with a larger cap is equivalent to extending the original path only if seeded replay is deterministic and the new cap does not change decisions before the old cutoff. Starting a fresh session is not enough to guarantee that. Equal numerical limits also do not prove equal codelet semantics or equal behaviour across numerical backends. A *backend* is the software-and-hardware route used to execute the calculation. The records include an ordinary CPU (central processing unit) path and a path using the MLX numerical library. They do not establish equivalence between CPU and GPU (graphics processing unit) execution.

B.4 The Earlier Checks With Memory

The historical learning-mode reference is described as 500 eight-run episodes per problem: 9,500 episodes, or 76,000 inner runs. A port check uses 100 episodes per problem: 1,900 episodes and 15,200 inner runs per cycle. The reference-to-check episode ratio is therefore 5:1, far below the 197:1 of the individual-run checks, though both references were meant for reuse.

Those 500 episodes do not establish the single-run heuristic target. Stored episodic singleton ratios reach 0.016, and the available record has no independent missing-mass validation of this reference.

What does memory change? The port retains descriptions of solutions and of *snags*, difficulties met while forming an analogy. Between runs it resets the workspace, the concept network, the codelet scheduler, and the current reminding activations, but keeps the stored experiences. The direct effect on later search is the rejection of a solution already remembered under the relevant structural description.

The duplicate check uses the problem, the solution string, the inferred rule, and the translation of that rule to the target, so the same letters reached through a different rule need not be a duplicate. Solution-finding, justification, and recovery from a justification clamp all consult this memory; the clamp is part of the control of justification, and its recovery path must not bypass the check. In the reference source, a solution already present in memory makes the solution-finding codelet *fizzle*: it stops without accepting the candidate. The port does the same. Search continues, which changes both what later runs find and whether they terminate.

There is also an explanatory role. After storing a solution, the inspected reporting code compares it with earlier solutions through rules, themes, justification, and coherence, then records reminding strengths and generates commentary. Here *themes* describe conceptual relationships in an analogy, and *coherence* is the program’s assessment of how well those relationships fit together. Stored snags can change how an otherwise unjustified theme is explained. These are implemented behaviours, not evidence that remembering improves

solution quality. The repeated-problem experiment tests exploration conditioned on experience, not learning transferable to new problems.

Existing regression definitions address several memory-related repairs: distinguishing solutions with different rules; preventing clamp recovery from accepting a remembered solution; comparing snag rules structurally rather than by potentially identical English descriptions; and assigning zero reminding strength at the relevant distance threshold. Further tests cover distance components, snag-based theme classification, clearing activations without deleting solutions, and session-scoped identifiers. The test driver counts a run as successful only if it reports a solution and adds a new memory entry.

These test definitions were inspected for this paper; the engine regressions were not freshly rerun, and Scheme and Python memory paths were not compared. Nor do the historical build records let us attribute each change in the endpoint counts to a particular memory repair.

For the saved port episodes, we recover the last solution found by looking backward past `*NONE*` and `*CAP*`, then check the stored endpoint counts and missing-head lists. Four episodes in each cycle produce no solution; the old endpoint histogram omits them but records their number, which Table 5 retains.

The historical episodic reference itself is absent from the supplement: its size comes from the development account, and the stored outside-solution lists cannot be regenerated. The saved port sequences do establish how often each listed endpoint occurred.

Table 5: Archived eight-run episodes. Novelty labels come from stored flags; the historical episodic reference is unavailable for independent re-computation.

Metric	CPU before	CPU after	Archived MLX
Check episodes	1,900	1,900	1,900
Within-episode runs	15,200	15,200	15,200
Stored novel endpoint pairs	17	13	17
Episodes with stored novel endpoint	28	14	28
Missing stored head members	0	0	0
Episodes never answering	4	4	4
Capped within-episode runs	1,311	2,404	1,309
Episodes with any capped run	628	1,052	626
Episodes ending in a capped run	361	591	361

The paired CPU records contain 17 outside problem/endpoint pairs across 28 episodes before repair, and 13 pairs across 14 episodes afterward. Neither cycle misses a selected head member. Of the 13 post-repair pairs, 11 also appear in the separate single-run reference. The other two are `misc2/ajd` and `eqe-baaab/qrrbq`, each seen once. That overlap is context only: an solution possible without memory need not have the same probability, or be reachable at all, under a particular learned context.

The development investigation links the direction-image repair to the disappearance of `abbbb`, `baaba`, `cdddb`, and `cddbc` from the episodic outside list, which complements the individual-run findings under the same limits on historical build evidence. The archived MLX sequences are another unvalidated cycle.

Two problems keep these records from being a controlled learning-mode validation. First, the port cap is 20,000 codelets and the reported reference cap is 100,000. An early cap can change memory and therefore every later run in the episode, and similar aggregate cap rates would not make that harmless: capped CPU runs rise from 1,311 to 2,404 after repair even as outside endpoints fall. Later prose reports 2,406 capped runs in 1,054 episodes; the identifiable saved sequences give 2,404 in 1,052, and we use the latter.

Second, episode e , run i uses seed $900000 + 8e + i$, with $0 \leq e < 100$ and $0 \leq i < 8$, so these paired samples overlap the single-run seed block and cannot confirm it independently. Those limitations led to the later study with matching caps and native winners.

C Single-Run Protocol and Additional Results

C.1 The Protocol and Its Change After a Failure

Every run starts with empty memory. Both programs receive a direct 100,000-codelet limit. For each of the 19 problems, we collect 20,000 reference runs to build the oracle, 30,000 separate reference runs for validation, and 1,000 port runs. The totals are 380,000, 570,000, and 19,000, respectively: 969,000 main observations.

Validation and port observations are divided into prespecified blocks of 100 runs, giving 300 reference checks and ten port checks per problem. The three phases use disjoint seed blocks. A seed equals the recorded phase base plus $100000j + i$, where j indexes the problem and i the run. Independence under a fixed distribution remains a modelling assumption about this pseudorandom sampling.

Reference execution uses Chez Scheme 9.5.4 in an emulated Linux/amd64 environment. The port runs serially using Python 3.14.6 and NumPy 2.5.1. Manifests identify the source, parameter files, dependencies, protocol, and completion receipts by their file hashes, allowing later checks to detect whether a file changed.

The original `support-v1` collection stopped when the reference failed reproducibly during validation. Rather than drop the failure or repair the engine mid-collection, we made a separately versioned continuation with the original budgets, problems, seeds, engines, caps, and frozen reference sets. A recognized failure inside the engine becomes `*ERROR*` and the process restarts at the next assigned seed; infrastructure and unclassified failures still stop collection.

Verified complete chunks from the parent study are inherited once. The interrupted chunk is replayed with its original seeds, checking that its 238 successful prefix observations agree. Exactly one observation is admitted for each assigned seed. Incomplete attempts are retained as records of what happened, not extra independent data.

The 969,000 main observations comprise the scheduled construction, validation, and port checks. No engine repair was introduced. To be clear: the continuation is an explicit response to a failure, not an unchanged experiment planned in every detail before it began.

C.2 The Reference Also Failed

The expensive reference validation exposed three failed executions on allowed inputs. Their seeds and failure signatures are retained:

Input	Seed	Codelets	Exception signature
<code>misc1</code>	20713988	2,835	Non-procedure #f
<code>misc1</code>	20716342	1,981	Non-procedure #f
<code>misc3</code>	20226148	2,088	<code>caddr: incorrect list structure</code> #f

Three executions, two exception signatures, and so far one diagnosis: the first failing continuation identifies a missing letter-category descriptor in `make-group`, and we do not yet know whether the other failures share that cause. Errors stay in the execution denominators and receive separate hard-error flags. The port has no engine errors in this study.

Finding these reference failures is another useful development result: building and validating a large oracle stress-tests the original program, and the retained failures are concrete starting points for repair and regression tests. Any sample of comparable size might have found them; we left them in the study.

C.3 Alternative Collection Rules and Their Costs

Because this study saves discovery order, we can ask what would have happened had construction stopped earlier. A *prefix* is the first part of the ordered sample. We compare fixed prefixes of 1,000, 5,000, 10,000, and 20,000 runs per problem, plus two stopping heuristics checked every 500 runs after at least 3,000 observations.

The singleton rule accepts $f_1/N \leq 10^{-4}$, requiring at least 10,000 runs when $f_1 = 0$. The no-discovery rule stops after a gap of at least 1,000 runs without a new outcome. If a rule never fires, it retains the full 20,000-run prefix.

The singleton rule fires for 14 problems. Including the five that reach the limit, it uses 268,500 construction observations and leaves 135 held-out outside draws and nine port outside draws. The no-discovery rule fires for all 19, uses 81,500 observations, and leaves 346 and 34 outside draws, respectively. Table 9 reports all six choices. They share observations and spend different amounts, so the comparison is descriptive; and since the full construction was collected anyway, the prefix totals are hypothetical allocations.

Construction and validation together cost 950,000 reference executions. The ten sets of port checks use 19,000 runs, a 50:1 aggregate count ratio; one benchmark-wide check uses 1,900, so the upfront reference is 500 times one check. The ten checks sample one frozen port. They demonstrate the allocation; savings across a development lifecycle would need successive revisions.

Recorded aggregate attempt wall time is about 466,573 seconds. It adds the durations of inherited, interrupted, continuation, and corrected preflight attempts. Because attempts may overlap, this sum is neither the campaign’s elapsed duration nor CPU time. Pilot work, earlier diagnosis, wrapper-development preflight, setup, human work, and analysis are not fully included. A complete cost claim would need those too.

C.4 All Outside Port Pairs

The following table retains every outside port pair from the frozen-reference check.

Table 7: All five flagged port input–outcome pairs. Held-out matches do not enlarge the frozen reference or remove recorded flags.

Input	Outcome	Port draws	Held-out reference draws
misc1	*NONE*	1	0
fig5.4-top	qqiqq	1	5
run6	pqqbc	1	2
copy5	acc	1	0
copy5	cbb	4	2

C.5 Individual-Run Tables

Table 8 reports all 19 problems in the later fresh-memory study. The construction sets remain frozen. Program errors remain in the validation and port denominators; an error can count both as an outside draw and as a hard error. Those columns therefore overlap. The three problems with no validation discoveries meet the nominal, individual 10^{-4} target, but none meets it under the 19-problem confidence allocation.

Table 8: Versioned single-run results. States counts outcomes in 20,000 construction runs. Reference validation uses 30,000 runs per input and the port 1,000; errors are included in new draws. Flag counts port 100-run checks with an outside draw or absent p50 member, out of ten checks per input.

Input	States	Ref. new	Ref. err.	Port new	Port err.	Flag
misc4	4	0	0	0	0	0
fig5.7	20	2	0	0	0	0

Input	States	Ref. new	Ref. err.	Port new	Port err.	Flag
misc3	58	15	1	0	0	0
misc5	8	0	0	0	0	0
misc2	9	1	0	0	0	0
run1	11	1	0	0	0	0
run4	12	1	0	0	0	0
misc1	17	4	2	1	0	1
fig5.4-top	41	28	0	1	0	1
eqe-baaab	41	17	0	0	0	0
run6	68	28	0	1	0	1
run3	13	6	0	0	0	0
run2	12	1	0	0	0	0
copy1	2	3	0	0	0	0
copy2	2	1	0	0	0	0
copy3	2	2	0	0	0	0
copy4	2	3	0	0	0	0
copy5	13	11	0	5	0	3
copy6	5	0	0	0	0	0

Table 9 compares the prespecified collection rules on prefixes of the same ordered construction samples, all 20,000 runs of which were collected; the simulated stops describe possible budgets. Each row uses the same validation and port observations.

The singleton rule fails to trigger on five inputs; their full prefixes and later discoveries remain included. The no-discovery rule triggers on every input but leaves more outside reference and port draws. No later discovery is merged into an earlier reference.

Table 9: Shared validation and port observations under ordered-prefix construction. Fired counts triggering inputs (out of 19); untriggered rules retain 20,000 runs. Construction budgets differ.

Construction rule	Build runs	Ref. new	Port new	Fired
Fixed 1,000/input	19,000	991	49	—
Fixed 5,000/input	95,000	383	33	—
Fixed 10,000/input	190,000	229	16	—
Fixed 20,000/input	380,000	124	8	—
Singleton heuristic	268,500	135	9	14
No-discovery heuristic	81,500	346	34	19

D Full Episodic Coverage and Port Results

Did the frozen references pass validation? Table 10 reports every problem, including failed and skipped validation. A and B mean quality and preference winners. “Build E ” counts construction episodes; “Freeze” gives the episode at which the corresponding set stopped changing. A dash means no freeze or no collected validation; zero misses is written 0. Five `copy5` validation episodes produce no solution, leaving 995 episodes with solutions for each population; every other tested population has 1,000.

Table 10: Episodic frozen-oracle coverage. E counts construction episodes; N_v counts validation episodes with solutions. A and B denote the native quality and conceptual-preference populations. Pass lists definitions meeting the 0.01 coverage target with 95% simultaneous confidence across all 38 populations. Dashes denote an unmet freeze or uncollected validation, not zero misses.

Input	Build E	Freeze A	Freeze B	N_v	Miss A	Miss B	Pass
<code>misc4</code>	2,000	—	1,000	—	—	—	not tested
<code>fig5.7</code>	8	8	6	1,000	37	111	none
<code>misc3</code>	11	11	7	1,000	0	99	A
<code>misc5</code>	2	2	2	1,000	480	591	none
<code>misc2</code>	2	2	2	1,000	6	63	none
<code>run1</code>	2,000	1,000	—	—	—	—	not tested
<code>run4</code>	2,000	—	1,000	—	—	—	not tested
<code>misc1</code>	14	5	14	1,000	589	193	none
<code>fig5.4-top</code>	2,000	—	—	—	—	—	not tested
<code>eqe-baaab</code>	2,000	2	—	—	—	—	not tested
<code>run6</code>	21	21	2	1,000	101	425	none
<code>run3</code>	2	2	2	1,000	377	418	none
<code>run2</code>	2	2	2	1,000	251	242	none
<code>copy1</code>	2	2	2	1,000	0	0	A,B
<code>copy2</code>	2	2	2	1,000	0	0	A,B
<code>copy3</code>	2	2	2	1,000	0	0	A,B
<code>copy4</code>	2	2	2	1,000	2	0	B
<code>copy5</code>	2	2	2	995	36	29	none
<code>copy6</code>	2	2	2	1,000	3	2	none

Only `copy1`, `copy2`, and `copy3` meet the limit of 1% outside solutions for both populations. Eight of the 28 tested populations pass: those six, `misc3 best_a`, and `copy4 best_b`. The other 20 fail. The ten populations belonging to five construction-limited problems receive no validation. Each passing population has zero misses among 1,000 episodes with solutions, giving an adjusted upper bound of about 0.006611 under the fixed 38-population allocation.

The early-stopping problem is especially clear in `misc5`. Both sets freeze after just two episodes. Yet validation then produces 480 out of 1,000 quality winners and 591 out of 1,000 preference winners outside those sets. The estimate was zero early, but the reference was far from adequately covered. Independent validation exposed that failure. Fewer construction runs here are not evidence that learning made oracle collection more efficient.

Could we have chosen a better population? Possibly. We chose quality and preference to make the episode’s selected result internally justifiable by Metacat, rather than simply using the last solution. We did not know that the resulting discovery frequencies would suit Good–Turing-guided collection. It may work more effectively with a different set of episodic results, and a future experiment could choose another conceptually meaningful projection to build a better oracle.

That needs a new construction sample and its own validation. The present data cannot separate the effect of the population choice from the effect of stopping immediately, and no change of definition would turn a zero singleton ratio into a confidence certificate. The failed validations stand as results to explain.

What did the port do? Table 11 gives both populations for every problem, validated or not, each with 100 assigned port episodes. Four `copy5` episodes produce no solution, so its winner denominator is 96; every other denominator is 100. “Outside” counts occurrences outside that population’s frozen set. Where one population froze and its partner did not, membership is reported descriptively for the frozen one.

Table 11: Port results for all 19 inputs, each assigned 100 eight-run episodes. N_p counts complete episodes with solutions; No solution counts episodes with no solution; Cap runs counts inner runs reaching the codelet limit (out of 800). Outside counts refer to independently frozen supports. A dash means no frozen support, not zero outside winners. Ref. pass repeats the independently validated reference populations; it is not a port pass/fail verdict.

Input	N_p	No solution	Cap runs	Outside A	Outside B	Ref. pass
<code>misc4</code>	100	0	0	–	0	not tested
<code>fig5.7</code>	100	0	0	2	13	none
<code>misc3</code>	100	0	66	24	26	A
<code>misc5</code>	100	0	0	47	62	none
<code>misc2</code>	100	0	406	0	8	none
<code>run1</code>	100	0	122	0	–	not tested
<code>run4</code>	100	0	0	–	0	not tested
<code>misc1</code>	100	0	97	24	36	none
<code>fig5.4-top</code>	100	0	0	–	–	not tested
<code>ege-baaab</code>	100	0	0	6	–	not tested
<code>run6</code>	100	0	0	7	33	none
<code>run3</code>	100	0	0	45	54	none
<code>run2</code>	100	0	189	23	28	none
<code>copy1</code>	100	0	233	0	0	A,B
<code>copy2</code>	100	0	215	0	0	A,B
<code>copy3</code>	100	0	224	0	0	A,B
<code>copy4</code>	100	0	229	0	0	B
<code>copy5</code>	96	4	0	2	3	none
<code>copy6</code>	100	0	216	0	0	none

Among the eight qualified populations, only `misc3 best_a` has outside port winners: 24 in 100 episodes. The other seven have none.

Large outside counts also appear against references that failed validation: for `misc5`, 47 quality and 62 preference winners outside in 100 episodes; for `run3`, 45 and 54. With no coverage guarantee behind those sets, the counts are observations only.

Selecting a solution and completing a run are different behaviours. The port attempts all 15,200 scheduled inner runs. Of these, 9,815 produce a solution, 3,388 stop without a solution below the cap, and 1,997 reach the cap. None has an engine error. Caps occur in 863 episodes, but another run in the same episode may still provide a winner. The four episodes with no solution supply neither A nor B and remain in the execution totals. Appendix F accounts for every phase and the inherited error episode. Comparing winners does not test whether the two systems produce a solution or terminate equally often.

E The misc3 Evidence in Detail

Table 12 contains every selected-solution count for `misc3`. Each construction column ends at its own freeze point, not the final common construction horizon. Reference validation does not add solutions to either frozen set. Counts are one selected string per complete episode with a solution, not all tied winners or all solutions across its eight runs.

Table 12: Complete `misc3` selected-solution counts. Freeze A and B use 11 and seven construction episodes, respectively; reference validation uses 1,000 episodes and the port check uses 100. Only `kji`, `kkjjii`, and `kkkjjjiii` belong to the frozen supports.

Solution	Freeze A	Freeze B	Ref. A	Ref. B	Port A	Port B
jii	0	0	0	1	0	0
kji	3	2	466	380	25	28
kjii	0	0	0	1	0	1
kjiii	0	0	0	8	5	2
kjji	0	0	0	3	0	1
kjjiii	0	0	0	1	1	1
kjjji	0	0	0	9	2	1
kjjjiii	0	0	0	7	4	1
kkji	0	0	0	2	0	0
kkjiii	0	0	0	1	0	0
kkjji	0	0	0	3	0	0
kkjjii	6	3	319	293	15	19
kkjjiii	0	0	0	4	0	0
kkjjiiii	0	0	0	2	0	0
kkjjjii	0	0	0	5	0	1
kkjjjii	0	0	0	3	0	0
kkjjjiii	0	0	0	3	0	1
kkjjjiiii	0	0	0	3	0	1
kkkji	0	0	0	9	2	4
kkkjiii	0	0	0	5	3	1
kkkjji	0	0	0	3	1	0
kkkjjii	0	0	0	2	0	1
kkkjjiii	0	0	0	10	4	7
kkkjjji	0	0	0	8	1	1
kkkjjjii	0	0	0	4	0	1
kkkjjjiii	2	2	215	228	36	27
kkkkjjji	0	0	0	2	1	1

What the saved descriptions say. All 24 outside port quality winners are also undefeated in the conceptual comparison. They are marked coherent, have three themes and zero unjustified themes, and score from 84 to 88. Unequal numbers of letters in the solution do not by themselves make it incoherent under the native checks.

In 20 of the 24 episodes, saved descriptions also establish that a solution belonging to the frozen set was available: 18 had lower quality, and two tied for highest quality. The other four lack saved evidence of an inside solution, and since the export does not retain every losing description, one may still have occurred.

The three tie cases explain the 21/1/2 grouping in Section 5.2. Using one-based port episode numbers: episode 43 ties `kjjjiii`, `kjjji`, and `kkkji` at quality 86, all outside. Episode 74 selects `kjjji`, tied with inside solution `kkkjjjiii` at 87. Episode 84 selects `kjjjiii`, tied with outside `kkkjjiii` and inside `kkkjjjiii` at 86. Selection uses original memory order, which the sorted exports do not preserve, so arrival order for the three ties cannot be independently recovered. Changing tie order cannot remove the other 22 outside memberships.

How the quality score is calculated. Both inspected code paths use $\text{round}(0.6Q_{\text{rule}} + 0.4(100 - T))$. Here Q_{rule} is the native rule-quality score and T is the program’s temperature, an internal control quantity rather than a physical temperature. Rule quality combines uniformity, abstractness, and succinctness: native assessments of consistency, generality, and economy of the rule. Sharing this outer formula does not establish that both programs supply the same inputs to it.

An illustrative calculation shows why rule details matter. At abstractness 96 and uniformity 100, a one-clause rule has succinctness 100 and rule quality 98; three unit-cost clauses have succinctness 67 and rule quality 84. At temperature 10, the resulting solution-quality scores are 95 and 86. Outside strings are not uniformly over-scored in the port, either: reference preference co-winners with string `kkkji` reach quality 88, whereas its two selected port quality winners score 86. The lack of high-scoring port episode maxima is a lead to investigate.

Two source differences, not two proved causes. First, when the port looks for changes shared by several components, its filter requires an explicitly recorded, non-null relation. The reference excludes only Identity (no change) and can retain a shared literal destination without a named relation. In plain terms, a rule may recognize a common change without expressing it as a named relation such as successor. A letter-to-group object-category change is relevant to grouping `a -> aa`, `b -> bb`, and `c -> cc` under one description. Excluding such schemas, or rule templates, can change which concise component-level rules are available.

Second, the port generalizes common schemas to subobjects without the reference checks for component coverage, shared enclosing objects, and descriptors on uncovered components. Those conditions limit when a change observed in components is eligible for generalization. Omitting them can admit proposals the reference would not make from the same cluster. A subsequent evaluator still assesses rules, however; the missing checks alone do not prove that an invalid proposal survives.

The inspected source hashes match the frozen study manifest and the reconstructed-reference manifest. A read-only audit of the full archive verified all 1,111 `misc3` completion receipts and their 9,699 named files against the public episode and selection records. The 1,011 reference raw exports agree with the published co-winner descriptions and saved earliest selections. This audit checks that the records agree with each other.

The archive lacks rule clauses, rule-quality components, workspace groupings, and a complete record of memory interventions, so outside episodes cannot be assigned to generation, scoring, translation, or memory causes. This investigation includes no repair, new episode, or enlarged oracle; checking a repaired port needs a separately versioned comparison. Without a matched memory-disabled control, whether learning caused the difference stays open.

F Episode Execution Counts

Keeping the denominators straight. Construction has 10,073 complete episodes with solutions and one error episode. No complete construction episode lacks a solution. Validation has 13,995 complete episodes with solutions and five with none. The port has 1,896 complete episodes with solutions and four with none. All nine complete episodes without solutions are on `copy5`. They count toward execution totals but supply no winner.

Table 13: Episodic inner-run counts, including the inherited pilot. The four disjoint outcome columns sum to Attempted; No solution excludes caps and errors.

Phase	Attempted	With solution	No solution	Capped	Error
Construction	80,591	65,108	13,467	2,015	1
Validation	112,000	73,383	20,132	18,485	0
Port	15,200	9,815	3,388	1,997	0

The inherited `run4` construction failure is zero-based episode 46, with first seed 90100368. It attempts seven runs: four produce a solution, two stop without one, and the seventh raises an unrecognized `get-bond-facet` message. The eighth is never attempted. Earlier solutions stay in the inner-run accounting, but the incomplete episode supplies neither A nor B. This unattempted eighth run explains 207,791 actual runs instead of $25,974 \times 8 = 207,792$ scheduled runs. The error was not retried as a replacement scientific observation.

Caps affect 1,489 construction, 7,405 validation, and 863 port episodes. These episode counts differ from the capped-run counts in Table 13, because one episode can contain several capped runs and still have a winner from another run.

G What the Supplement Can Reproduce

The submission attachment contains a standalone LaTeX source bundle for this manuscript and a separate experimental bundle. The latter contains historical measurements, scientific exports from both later studies, the recorded port source and seed data, analysis code, tests, and Metacat reconstruction patches. Its README links results to evidence and gives dependency instructions. Its manifest records included file hashes, original source hashes, replaced documentation, and exclusions.

The original Metacat source archive and runtime binaries are not redistributed. The reconstruction helper downloads the linked upstream release, applies the supplied patches, and checks the resulting files. This reconstruction supplies the modified reference without silently replacing its original licence or source distribution.

Checking saved data does not repeat the experiments. From the extracted experimental bundle, `python3 verify.py` checks file and frozen-source hashes, regenerates saved-data audit reports and tables, and runs the academic and reconstruction-helper tests. It also reproduces episodic stopping decisions, validation eligibility, bounds, and outcome counts. These checks use the Python standard library and run neither analogy engine. A Unix locking dependency requires macOS, Linux, or a Linux environment such as Windows Subsystem for Linux (WSL).

The historical audits recover p50 heads, stored flags, probability calculations, endpoint counts, and cap accounting from saved measurements. They do not recover missing discovery order, identify all sampled historical builds, or reconstruct historical episodic novelty from an unavailable reference.

The episodic all-problem audit checks both winner populations and their separately frozen sets against the saved episodes. It retains all 38 construction/port count vectors. The `misc3` audit records selected-solution counts, the 24 outside quality-winner events, native winner descriptions, and the diagnostic groups. It does not rebuild unrecorded rule clauses, workspace states, or causal interventions. The public episodic export contains observations and descriptions of selected and tied winners; the full attempt archive is private. The raw-record inspection described in Appendix E is consequently a documented audit finding that the public export alone cannot reproduce.

Four single-run scientific archives preserve ordered observations, raw attempt evidence, receipts, manifests, and collection history. The main observation ledger identifies each counted result; archived copies of attempts add no observations. The lightweight table audit checks saved summaries without repeating the 969,000 engine executions, the manuscript source bundle supplies this version’s tables, and optional reference reconstruction runs no analogy episodes. Checking the reported results needs neither the graphical interface nor new experiments.

For anonymous review, two repository READMEs have replacement instructions. Repository ignore files, a GUI screenshot, private operational metadata, and Git history are omitted. Scientific records, executable source, technical implementation labels, and hash chains remain unchanged. Third-party attribution, the Metacat licence, and the port’s MIT terms are retained; the MIT copyright holder’s display name is anonymized for review.

The bundle is neither a complete user-interface/database deployment nor a ready-configured rerun of the campaign, and packaging changed nothing in the studies.

H Notation

Table 14: Symbols keep observed counts, unknown probabilities, and estimates separate.

Symbol	Meaning
$\mathcal{X}, \mathcal{Y}$	Countable sets of allowed analogy problems and finite letter-string solutions
$\mathcal{R}$	Problem–solution relation under fixed program settings and initial memory
p_R, p_P	Reference and port outcome probabilities under fixed settings
N_x, n_x	Reference and check sizes for problem x , in runs or episodes as specified
K, C_R	Number of check cycles and one-time reference cost
$C_{P,k}, C_{F,k}$	Cost of check k and of investigating its findings
L, ϕ	Runs per episode and the rule selecting an episode’s recorded result
J	Number of problems, 19 here
$c_x(o), f_{1,x}$	Count of outcome o and number of types seen exactly once
S_x, H_x, T_x	Observed reference support set for problem x , p50 head, and observed port-check set
$M_{R,x}, \widehat{M}_{R,x}$	True reference missing mass and its singleton-ratio estimate
$q_{P,x}$	Port probability of an outcome outside the frozen reference set
D_x, U_x	Outside occurrences and distinct outside outcome types

I The Nineteen Problems

Table 15 gives the complete benchmark. For example, the row’s Initial and Modified strings specify the demonstrated change, and Target is the string to which an analogous change is sought. The shared-solution column lists up to three outputs seen in both the historical reference and post-repair CPU check, ordered by decreasing reference count, with lexicographic ties.

Table 15: Benchmark inputs and observed shared solutions; examples are not correctness labels.

Input	Initial	Modified	Target	Shared examples
misc4	a	b	z	b, z, y
fig5.7	aabc	aabd	ijkk	ijll, ijl, ijl
misc3	abc	aabbcc	kkjjii	kkjjii, kkkjjjiii, kji
misc5	abc	abd	glz	gld, hlz, glz
misc2	abc	abd	ijk	ijl, ijd, ijk
run1	abc	abd	mrrjjj	mrrkkk, mrrjjk, mrrjjj
run4	abc	abd	xyz	xyd, wyz, xyz
misc1	abc	cba	mrrjjj	jjjrrm, mrrjjj, jrrmmm
fig5.4-top	eeqee	qeeq	xxixx	qeeq, ixxi, ixxq
eqe-baaab	eqe	qeq	abbba	baaab, qeeeq, qbbbq
run6	eqe	qeq	abbbc	qeeeq, baaaq, qcccb
run3	rst	rsu	xyz	xyu, wyz, xyz
run2	xqc	xqd	mrrjjj	mrrkkk, mrrjjk, mrrjjj
copy1	ab	c	ab	c, ab
copy2	bc	d	bc	d
copy3	xy	z	xy	z
copy4	zy	x	zy	x, zy
copy5	aabb	cc	aabb	cc, cac
copy6	abc	d	abc	d, abc

J Further Related Work and Applications

The difficulty of knowing what a test should expect is the *oracle problem* in software testing (Weyuker, 1982; Barr et al., 2015; Patel & Hierons, 2018). Comparing with another program is an established response: it can act as a pseudo-oracle (Davis & Weyuker, 1981), or as the reference in differential testing (McKeeman, 1998). Agreement does not prove both programs correct, and disagreement needs interpretation. Here the variation is intentional, unlike test *flakiness*, where repeated tests unexpectedly change their pass/fail result (Luo et al., 2014).

Another response is *metamorphic testing*: check a relationship between executions when individual expected outputs are hard to specify (Chen et al., 1998; 2018; Segura et al., 2016), for example when a known relationship between two inputs implies one between their outputs. Statistical versions and work on stochastic optimization address random outcomes (Guderlei & Mayer, 2007; Yoo, 2010). Such tests could complement ours where domain knowledge supplies reliable relationships. Tests of randomized algorithms also need a stated null hypothesis (the assumption the test is designed to challenge), a sampling protocol, and a specified difference to detect (Arcuri & Briand, 2014). Statistical model checking asks whether particular properties of stochastic executions hold (Younes & Simmons, 2002; Legay et al., 2010). Its guarantees do not come from repeatedly looking at a convenient statistic.

Good–Turing is well established (Good, 1953). Practical frequency estimation (Gale & Sampson, 1995), unseen-vocabulary estimation (Efron & Thisted, 1976), estimating the number of types (Chao, 1984), and coverage-based estimation (Chao & Lee, 1992) address neighbouring, non-interchangeable questions. Bounds on missing mass carry uncertainty terms (McAllester & Schapire, 2000), and the optimality results of Orlitsky et al. (2016) concern predicting numbers of unseen types rather than a stopping rule or port diagnostic.

The closest software-testing precedent is STADS (Böhme, 2018), which treats program behaviours as species to be discovered through sampling. Work on residual risk in greybox fuzzing, which uses execution feedback to generate tests, also estimates the chance of further discoveries (Böhme et al., 2021). Capture–recapture methods have been used to estimate software fault content (Eick et al., 1992). We therefore do not claim to have invented the use of discovery statistics in testing. Our focus is a frozen reference record that repeatedly supplies checks of named solutions during development. Guarantees for other sampling schemes do not automatically carry over.

Selecting the most frequent outcomes until a target share is reached is an empirical smallest covering region (O’Neill, 2022), which we apply at 50% without claiming a new algorithm or an optimal threshold.

Work on aligning and replicating simulation models distinguishes different levels of agreement (Axtell et al., 1996; Wilensky & Rand, 2007). Our distinction between solution sets and internal processes belongs in that tradition. Differential testing of probabilistic programming systems (Dutta et al., 2018) is a related application we have not tried. Resetting probabilistic stateful systems between fixed-length trials is established (Gerhold & Stoelinga, 2018); that is the relevant precedent for treating an episode as one observation. Missing-mass estimation for Markov sequences (Pananjady et al., 2024) studies dependence under explicit assumptions. Those results do not automatically justify pooling runs across an evolving memory history.

Where, then, is the process contribution? It joins construction, freezing, fast checks, and investigation into one reusable testing process for a stochastic port: Good–Turing guides reference collection, p50 heads name common reference solutions missing from a check, and the named differences guide code inspection and repair. We have not found this complete process in the prior work we examined. That is a bounded comparison, not proof of exclusive priority.

The learning-mode application is part of this contribution, with a boundary: the single-run studies and the historical episodic checks use p50 heads, while the later quality-and-preference study extends construction and independent coverage validation to two populations of selected episodic solutions and tests frozen support sets only.

J.1 Other Proposed Applications

Consider speeding up a randomized search program without changing the kinds of solutions it finds. An extensively sampled, reviewed version supplies the reference. Before touching the search, define the outcome categories, execution limits, and check budget; let Good–Turing guide collection and independent validation assess coverage. After each change, a budgeted sample reports unexpected solution types and missing common ones, giving the developer specific cases to investigate while unit tests check individual rules.

The same pattern could apply when restructuring a simulator or re-implementing a learning system while preserving selected behaviour. For a system that learns during testing, define the initial state and the unit of observation first; an episode-level result based on the system’s own assessments is one option, and it keeps the check on the behaviour of interest without a fixed solution per run or a comparison of internal sequences.

One distinction from full TDD matters. A sampled reference describes observed behaviour and says nothing about capabilities absent from it. For new behaviour, developers still need a requirement, a trusted example, or a relationship between executions; reference-based checks then guard the behaviour that should stay unchanged while those tests drive the addition. An intentional change in outcomes calls for a reviewed new reference.